INFORME DE DESARROLLO: SISTEMA DE GESTIÓN DE PARQUEO

Universidad de San Carlos de Guatemala (USAC) Juan Esteban Ruiz Villatoro 202400087

30 de octubre de 2026

1. Introducción

El presente informe detalla el ciclo de desarrollo del Sistema de Gestión de Parqueo, abarcando la planificación por etapas, la trazabilidad del avance técnico, los principales problemas de compilación y lógica encontrados durante la implementación, así como las soluciones aplicadas para cumplir con las restricciones académicas de estructuras manuales y concurrencia.

2. Plan de Implementación por Etapas

El desarrollo del proyecto se estructuró de manera modular en cuatro etapas secuenciales:

1. **Etapa 1: Diseño y Estructuras Base (Modelo y Estructuras)**
   - Creación de las entidades de dominio (`Residente`, `Vehiculo`).
   - Implementación manual desde cero de las estructuras de datos dinámicas: `ListaDobleResidentes`, `ListaSimpleVehiculos`, `ListaListasParqueo`, `ColaVehiculos`, `ColaSalida` y `PilaVehiculos`, evitando por completo el uso de colecciones de Java.
2. **Etapa 2: Interfaz Gráfica (Vista y Componentes Swing)**
   - Desarrollo de la ventana principal (`VentanaPrincipal`) y paneles de gestión (residentes, vehículos, simulación de garitas y eventos) utilizando Java Swing y AWT.
3. **Etapa 3: Concurrencia y Hilos (Control de Garitas)**
   - Implementación de los hilos de control (`HiloGarita` y `HiloGaritaSalida`) para simular de forma concurrente el ingreso y egreso de vehículos.
4. **Etapa 4: Validación Dinámica y Documentación**
   - Integración de la validación estricta de socios para el Área A–C y generación de los manuales técnicos y de usuario en formato LaTeX.

---

3. Registro del Avance

- **Fase Inicial:** Estabilización de las estructuras de nodos enlazados y pruebas unitarias de inserción y búsqueda de residentes.

- **Fase Intermedia:** Conexión de la interfaz gráfica con las estructuras de datos y renderizado dinámico de los 150 espacios de parqueo en parrillas independientes para áreas de socios (A–C) y general (E–I).
- **Fase Final:** Sincronización de hilos en las garitas de entrada/salida y depuración de llamadas a métodos de compatibilidad entre la vista y las clases de datos.

---

4. Problemas Encontrados y Soluciones Aplicadas

Durante el desarrollo se identificaron y resolvieron diversos inconvenientes técnicos:

1. **Incompatibilidad en nombres de métodos y tipos de nodos:**
   - *Problema:* Al intentar recorrer la lista doble de residentes desde `VentanaPrincipal`, el compilador arrojaba errores por tipos de datos incorrectos en los nodos y métodos no definidos (como `getPrimero()` o `getSocio()`).
   - *Solución:* Se ajustó la referencia utilizando la clase interna estática `ListaDobleResidentes.NodoRes` recorriendo con `getCabeza()` y consultando el estatus mediante el método nativo `isEsSocio()` junto con la validación de placas en la lista simple de vehículos del residente (`esVehiculoDeSocio(placa)`).
2. **Validación de áreas exclusivas (A–C):**
   - *Problema:* La validación inicial dependía de placas quemadas (hardcoded) y no reflejaba los cambios hechos dinámicamente en el panel de gestión de residentes.
   - *Solución:* Se integró la consulta directa al método `esVehiculoDeSocio(placa)` provisto en la estructura de residentes, permitiendo validar en tiempo de ejecución si el vehículo ingresado de forma manual o por hilo pertenece efectivamente a un socio registrado.
3. **Restricciones de arquitectura:**
   - *Problema:* Mantener el cumplimiento estricto de no utilizar librerías externas ni estructuras de la API de Java (`ArrayList`, `LinkedList`).
   - *Solución:* Se validó que todas las operaciones de almacenamiento y control de flujo utilicen exclusivamente los punteros y nodos programados manualmente.